

Seq2Seq & Attention

Hrachya Asatryan

Many-to-Many Problems

In the previous lecture, we focused on Many-to-One problems (e.g., Sentiment Analysis), where an entire variable-length sequence is compressed into a single label.

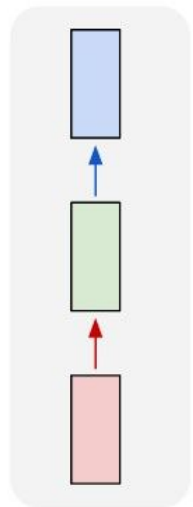
However, many fundamental language tasks require generating an **output sequence** of tokens.

We categorize sequence tasks into three distinct structural paradigms based on how input steps line up with output steps:

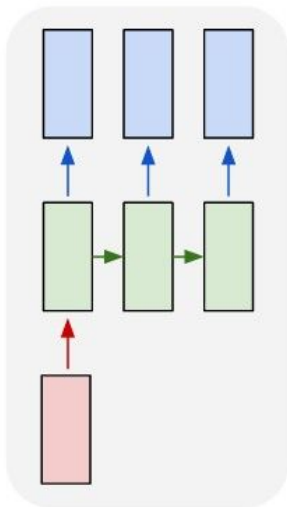
1. Synced Many-to-Many (Sequence Tagging)
2. Autoregressive Language Modeling (Causal Prediction)
3. Unsynced Many-to-Many (Sequence-to-Sequence / Seq2Seq)

Many-to-Many Problems

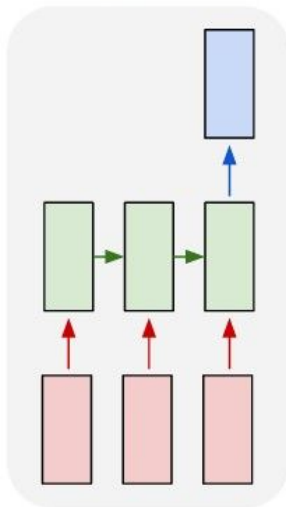
one to one



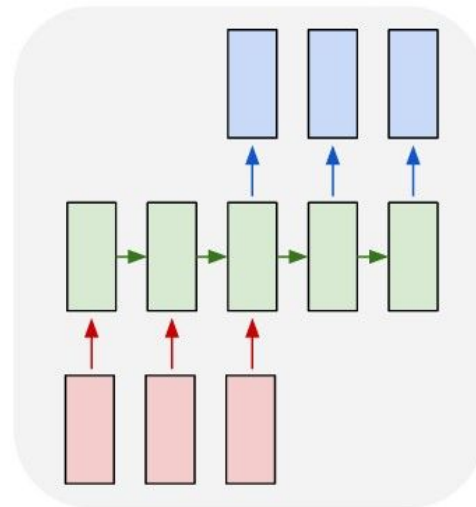
one to many



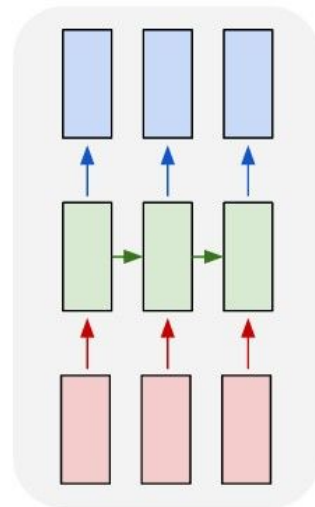
many to one



many to many



many to many



Many-to-Many Problems

1. Synced Many-to-Many (Sequence Tagging)

- The input length equals the output length ($T_{in} = T_{out}$).
- Every input step x_t generates a hidden state h_t , which immediately predicts an output tag y_t .
- *Examples:* Named Entity Recognition (NER), Part-of-Speech (POS) tagging.

2. Autoregressive Causal Language Modeling

- Predicting the next word given the preceding context.
- Formally computes the joint probability of a sequence as a product of conditional probabilities: $P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t | x_1, x_2, \dots, x_{t-1})$
- At time step t , the input is x_t , and the target label is x_{t+1} .

Many-to-Many Problems

3. Unsynced Many-to-Many (Sequence-to-Sequence)

What happens when the input sequence length and output sequence length are completely different? ($T_{in} \neq T_{out}$)

- **Machine Translation:** "I love deep learning" (4 tokens) -> "Ես սիրում եմ խորը ուսուցումը" (5 tokens).
- **Text Summarization:** A 500-word news article -> A 20-word summary.
- **Dialogue / Chatbots:** A 15-word user question -> A 50-word detailed response.

In these tasks, token 1 of the input does not map to token 1 of the output. We cannot generate output in lockstep with the input.

We need an architecture that reads the *entire* input sequence first before generating the first word of the output.

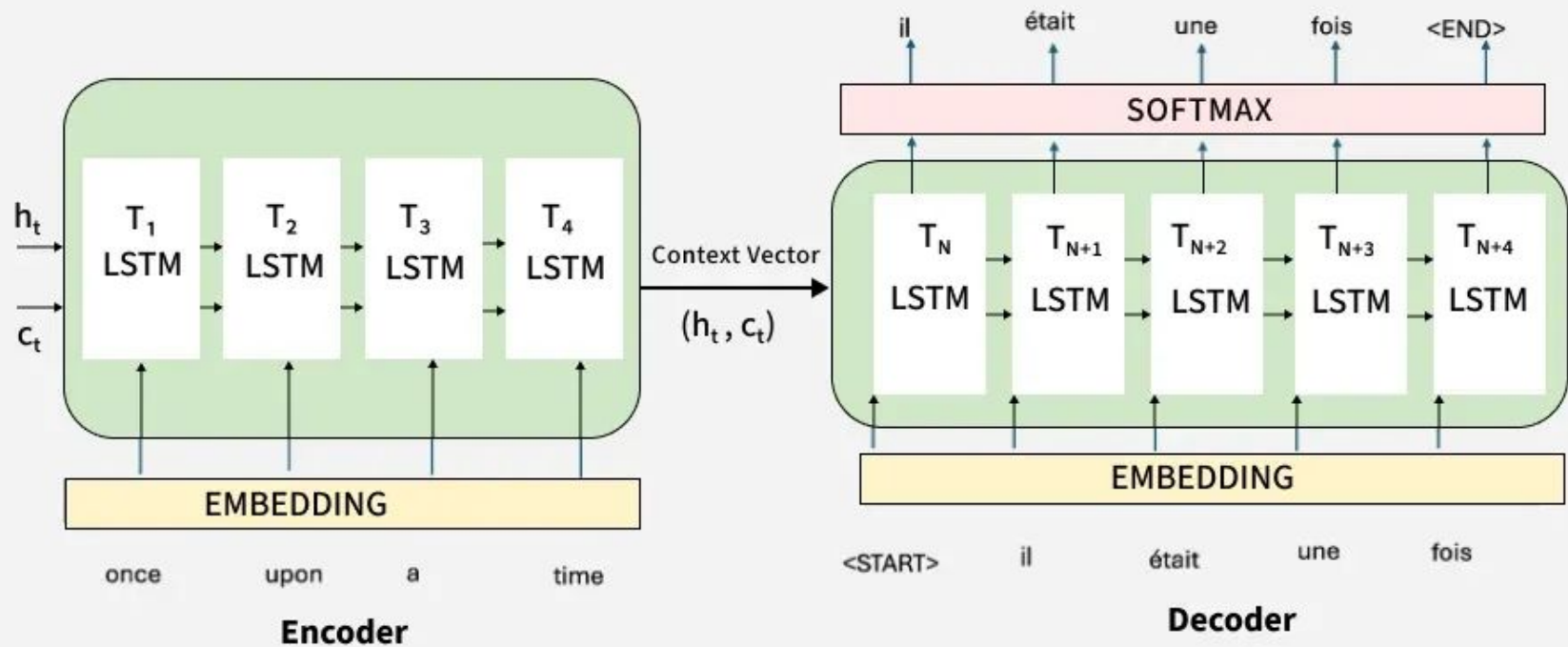
Many-to-Many Problems

The Encoder-Decoder Framework To solve unsynced sequence generation, Cho et al. (2014) and Sutskever et al. (2014) introduced the **Encoder-Decoder architecture**.

It splits the workload between two distinct neural networks:

1. **The Encoder:** An RNN/LSTM that reads the input sequence $(x_1, x_2, \dots, x_{T_{in}})$ step-by-step. Its sole purpose is to process the input and produce a final summary vector: $v = h_{T_{in}}$ (The Context Vector).
2. **The Decoder:** A second, distinct RNN/LSTM initialized with the context vector ($s_0 = v$). It generates the target sequence $(y_1, y_2, \dots, y_{T_{out}})$ autoregressively, one word at a time, until it outputs a special **<EOS>** (End of Sequence) token.

Many-to-Many Problems



Many-to-Many Problems

Decoder Execution: Training vs. Inference

How does the Decoder generate text?

- **Inference (Testing): Autoregressive Decoding** At step t , the model feeds its *own prediction* from step $t-1$ back into the network as the input for step t . *Risk*: If the decoder makes a mistake at step 1, that error cascades down the entire sentence.
- **Training: Teacher Forcing** Instead of feeding the model's prediction from step $t-1$, we feed the **ground-truth target token** y_{t-1} as the input to step t .

Why Teacher Forcing? Without it, early in training, the decoder outputs random nonsense at step 1, causing the entire sequence down the line to blow up and making gradient descent fail to converge.

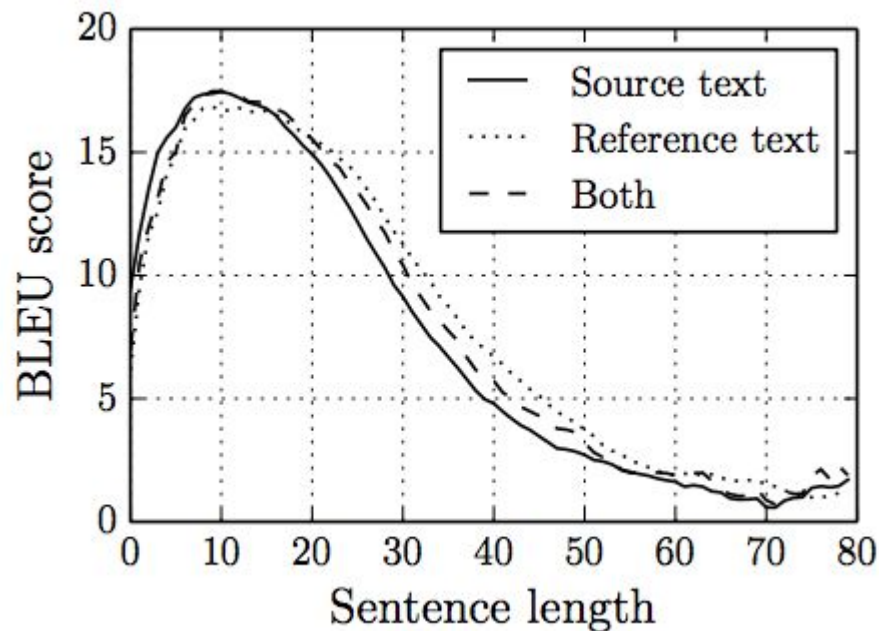
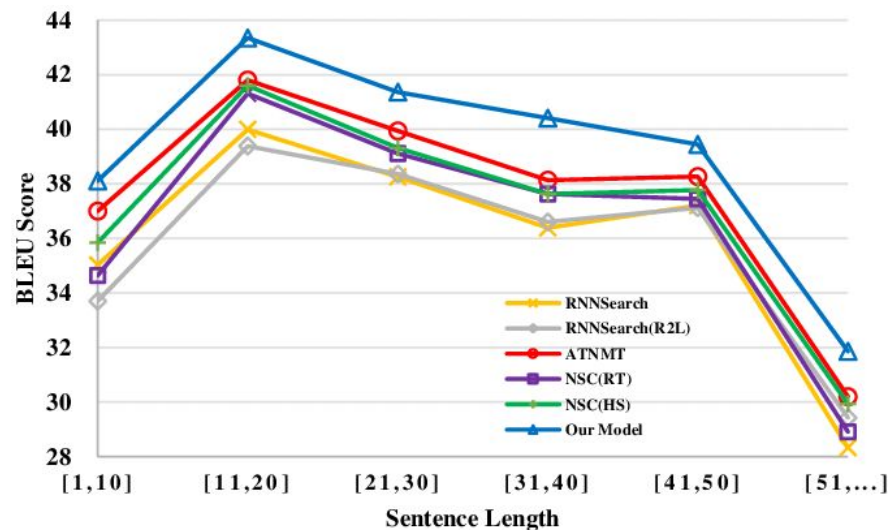
Many-to-Many Problems

- Stanford Lecture on Language Modeling
- Stanford Lecture on Machine Translation

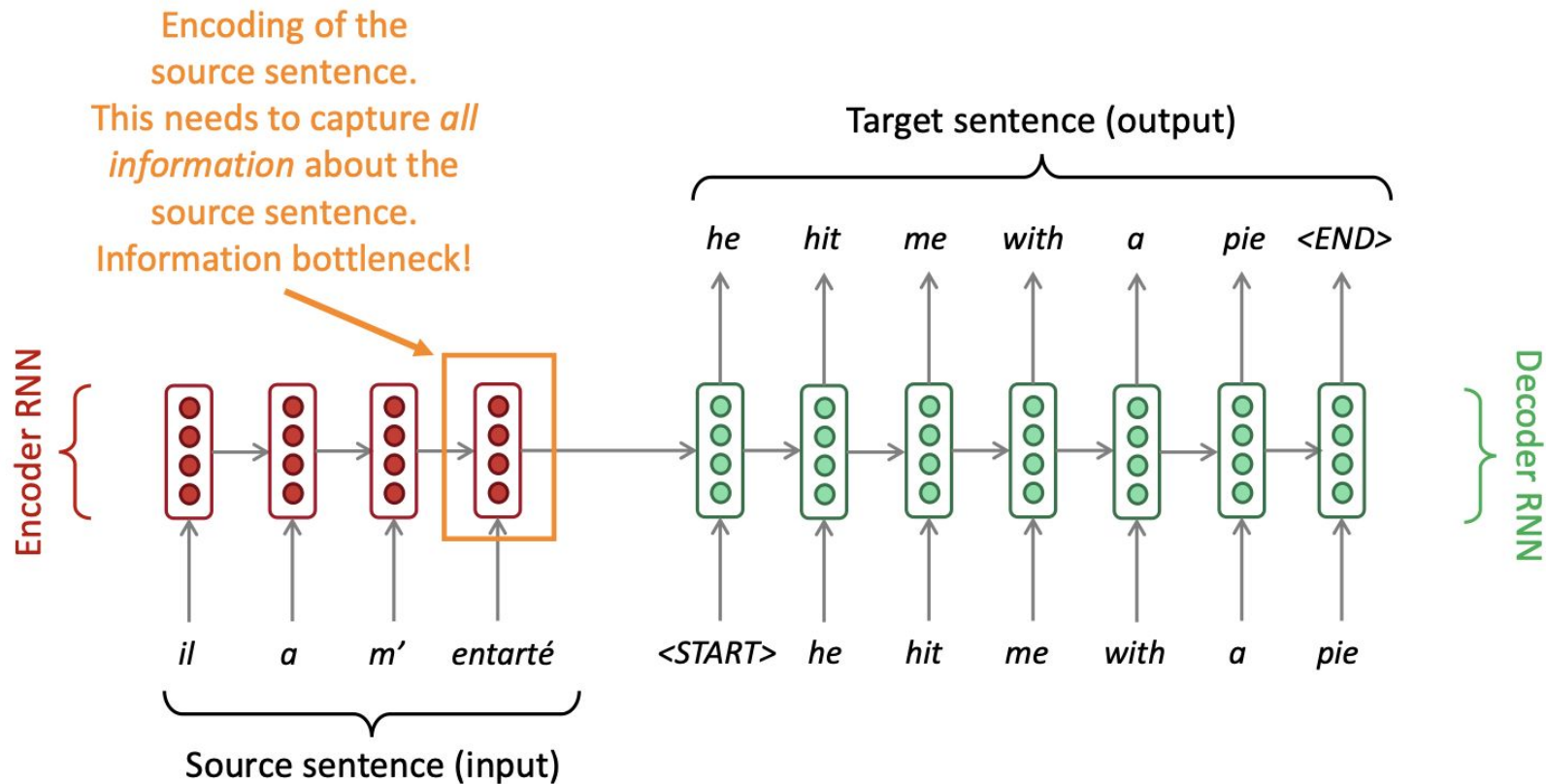
The Information Bottleneck

- Look closely at the Context Vector c . It is a fixed-size mathematical vector (e.g., 256 or 512 dimensions).
- **The Problem:** We are forcing the Encoder to compress the meaning, syntax, and nuance of an *entire paragraph* into a single 512-dimensional array.
- **The Result:** The model suffers from severe amnesia on long sentences. The BLEU score (translation accuracy) plummets for sentences longer than 20 words. You simply cannot compress that much data without loss.

The Information Bottleneck



The Information Bottleneck

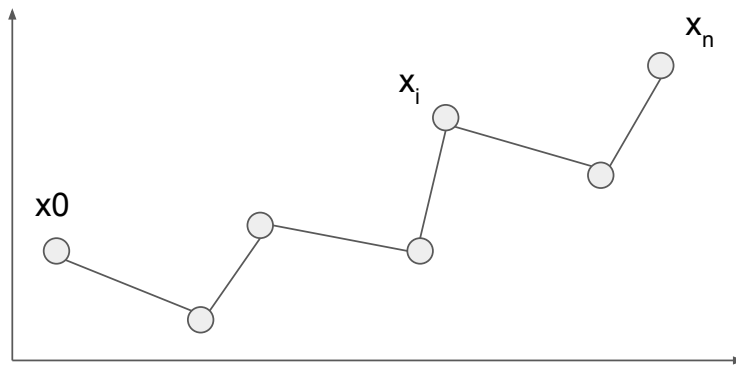


Breaking the Bottleneck

- **The Flaw of Seq2Seq:** The network reads the whole sentence, compresses it into one vector, and then throws away the individual word memories.
- **The Proposed Fix:** What if we *don't* throw away the intermediate hidden states? What if we keep all of them: $h_1, h_2, \dots, h_3$?
- **The New Problem:** If the network has access to 100 different word states at once, how does it know which ones are actually important for the current word it is processing?
- To answer this, we are going to leave NLP entirely for a moment and look at signal processing.

A Tangent into Time-Series

- Imagine a simple 1D time-series dataset: a stock price or an audio signal.
- Let our data be a sequence of points: $x = x_0, x_1, \dots, x_n$.
- **The Problem:** The signal is incredibly noisy. If we look at a single data point x_i in isolation, we can't tell what the true underlying trend is.
- **The Question:** How do we reduce the noise and increase the true signal for a specific point x_i ?



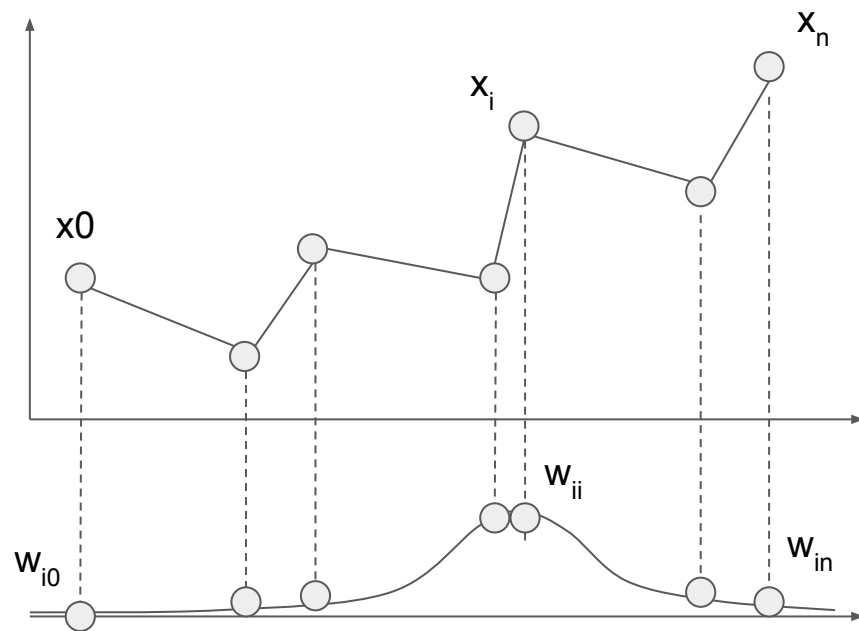
Self-Attention

- In classical statistics, we solve noise by taking a **weighted average** of the surrounding points.
- To calculate a new, "smoothed" value y_i for our target point, we look at its neighbors:

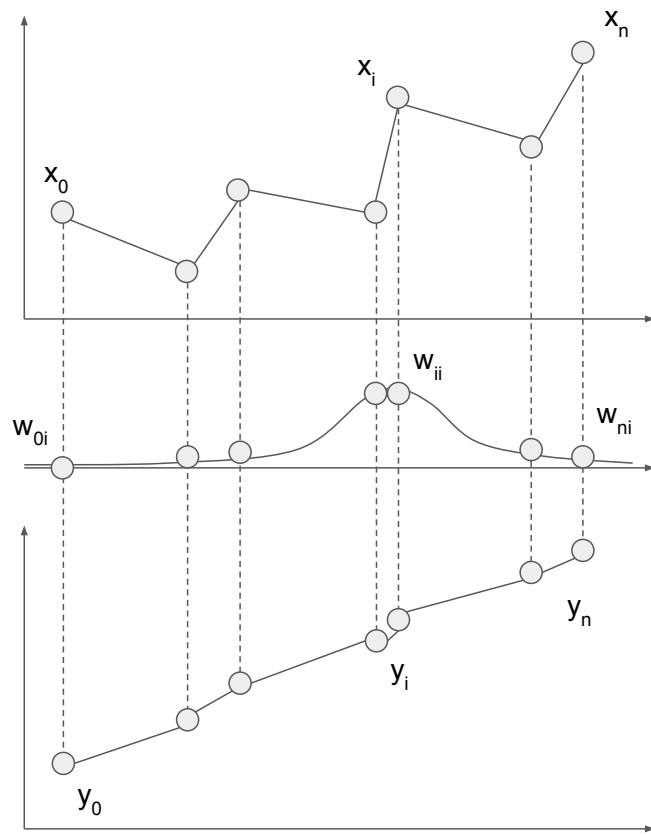
$$y_i = \sum w_{ij} x_j$$

- The weights w_{ij} are determined by a Gaussian filter based on *proximity*. If a point x_j is right next to x_i , its weight is near 1.0. If it is far away, its weight is near 0.0.
- *Crucial Concept*: The new point y_i is a mixture of itself and the context of its neighbors. It has "paid attention" to its surroundings.

Self-Attention



Self-Attention

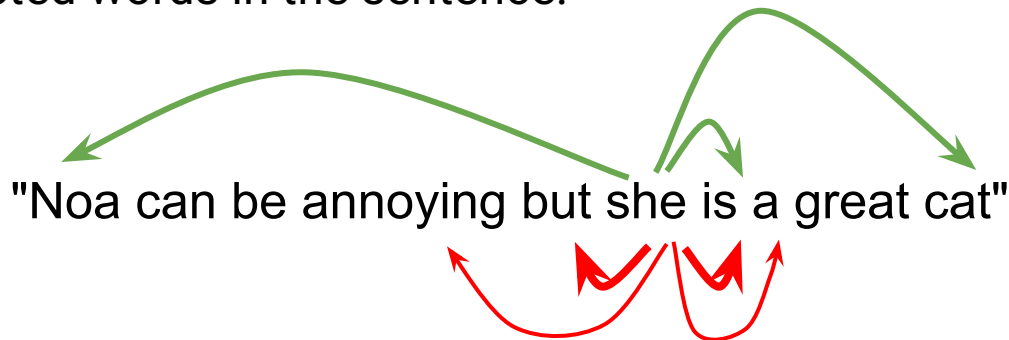


Self-Attention

- Human language is just a time-series of discrete tokens!
- A raw word embedding (like GloVe) is like a noisy data point. It has multiple potential meanings.
- *Example:* Look at the word vector for "**bank**". Is it a financial institution, or the side of a river? In isolation, it is noisy.
- Just like in the graph, we need to "smooth" the word "bank" by averaging it with the words around it to clarify its true meaning.

Self-Attention

- In a time-series graph, **physical distance = relevance**.
- In language, this is completely false! Semantic relationships do not care about physical proximity.
- *Consider this sentence:* "The **cat**, who already ate all of the expensive wet food we bought yesterday, is still **hungry**."
- "Cat" and "hungry" are 14 words apart. A proximity-based filter would give them a weight of 0.0. Yet, syntactically, they are the most deeply connected words in the sentence.



Self-Attention

- We want to compute a new, context-aware vector y_i for every word in the sentence.

$$y_i = \sum a_{ij} x_j$$

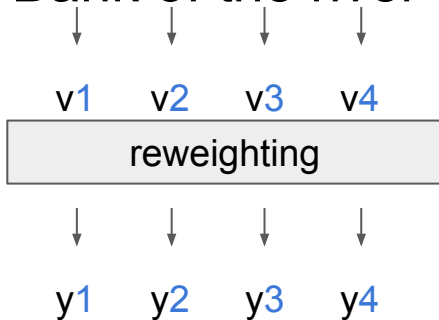
- **The Paradigm Shift:** Instead of hard-coding the weights (a_{ij}) based on physical distance, what if we let the neural network *calculate* the weights dynamically based on semantic meaning?
- Every word looks at every other word in the sentence. It asks:
"Mathematically, how relevant are you to my current meaning?"
- It converts those relevance scores into a probability distribution (where all weights sum to 1).

Self-Attention

- Let's process the sentence: *"The bank of the river."*
- The network calculates the relevance weights for the target word **"bank"**:
 - a. $\alpha_{\text{bank, river}} = 0.85$
 - b. $\alpha_{\text{bank, the}} = 0.10$
 - c. $\alpha_{\text{bank, of}} = 0.05$
- The new output vector y_{bank} is mostly made up of the vector for "river", slightly mixed with "the" and "of".
- **The Result:** The word "bank" has dynamically updated its own math to represent a water-bank, entirely based on its sequence. **The bottleneck is broken.**

Self-Attention

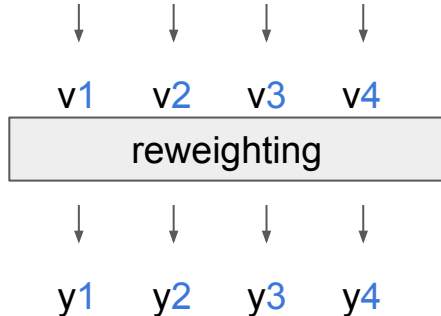
Bank of the river



y-s should have
more context

Self-Attention

Bank of the river



y-s should have more context

$$w^{11}v_1 + w^{12}v_2 + w^{13}v_3 + w^{14}v_4 = y_1$$

e.g. the contextualized vector for 'Bank' will have the features associated with 'river' enhanced because it is in the same sentence with river, and the similarity between Bank and river is high. So the contextualized embedding for Bank attended to river more than to the other words in the sentence.

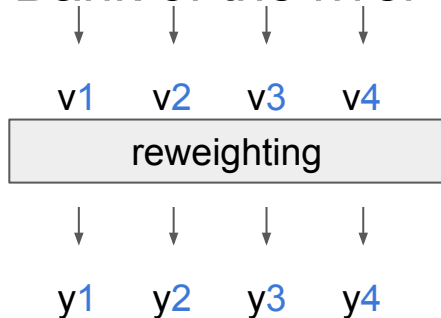
$v_1 v_1 = w^{11}$ normalize to
 $v_1 v_2 = w^{12}$ sum to 1 (e.g.
 $v_1 v_3 = w^{13}$ softmax)
 $v_1 v_4 = w^{14}$

w^{11}
 w^{12}
 w^{13}
 w^{14}

w_1

Self-Attention

Bank of the river



$$\begin{aligned}w_{11}v1 + w_{12}v2 + w_{13}v3 + w_{14}v4 &= y1 \\w_{21}v1 + w_{22}v2 + w_{23}v3 + w_{24}v4 &= y2 \\w_{31}v1 + w_{32}v2 + w_{33}v3 + w_{34}v4 &= y3 \\w_{41}v1 + w_{42}v2 + w_{43}v3 + w_{44}v4 &= y4\end{aligned}$$

So far:

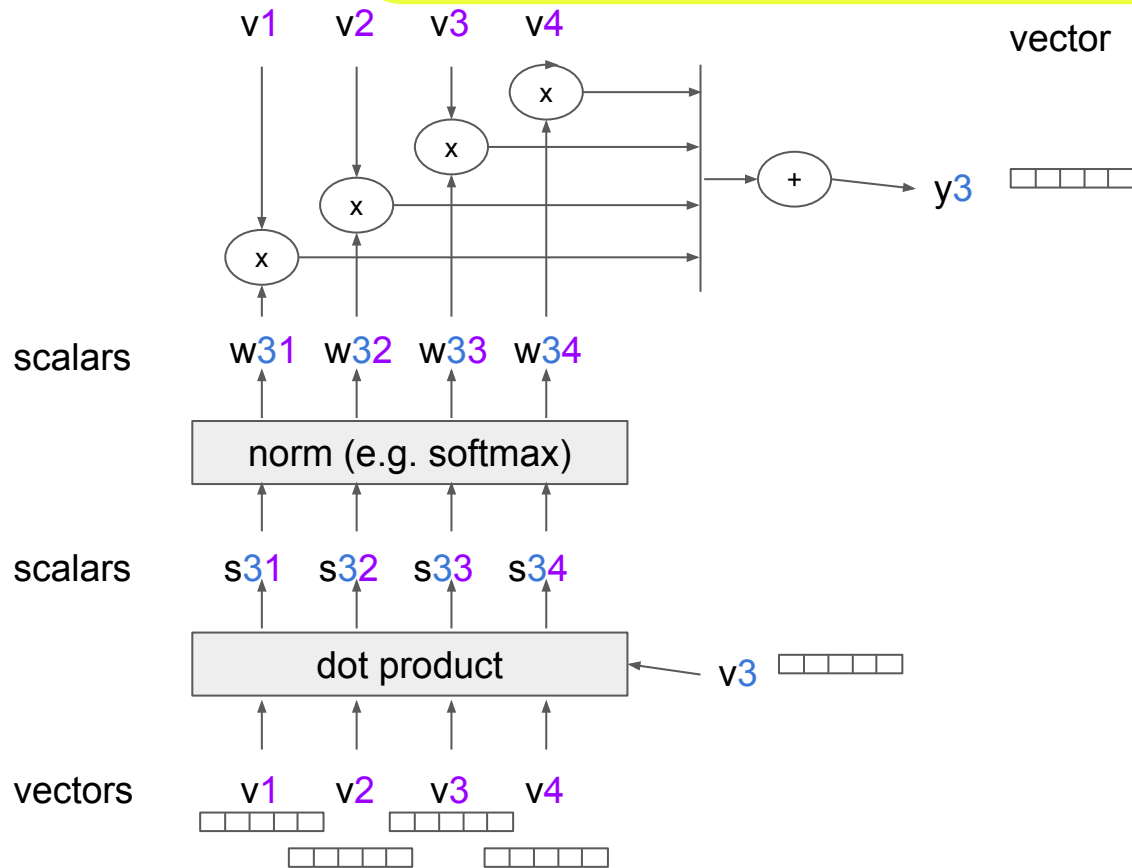
- No weights are trained
- The order does not matter
- The proximity does not matter

$v1 v1 = s11$ normalize to
 $v1 v2 = s12$ sum to 1 (e.g.
 $v1 v3 = s13$ softmax)
 $v1 v4 = s14$

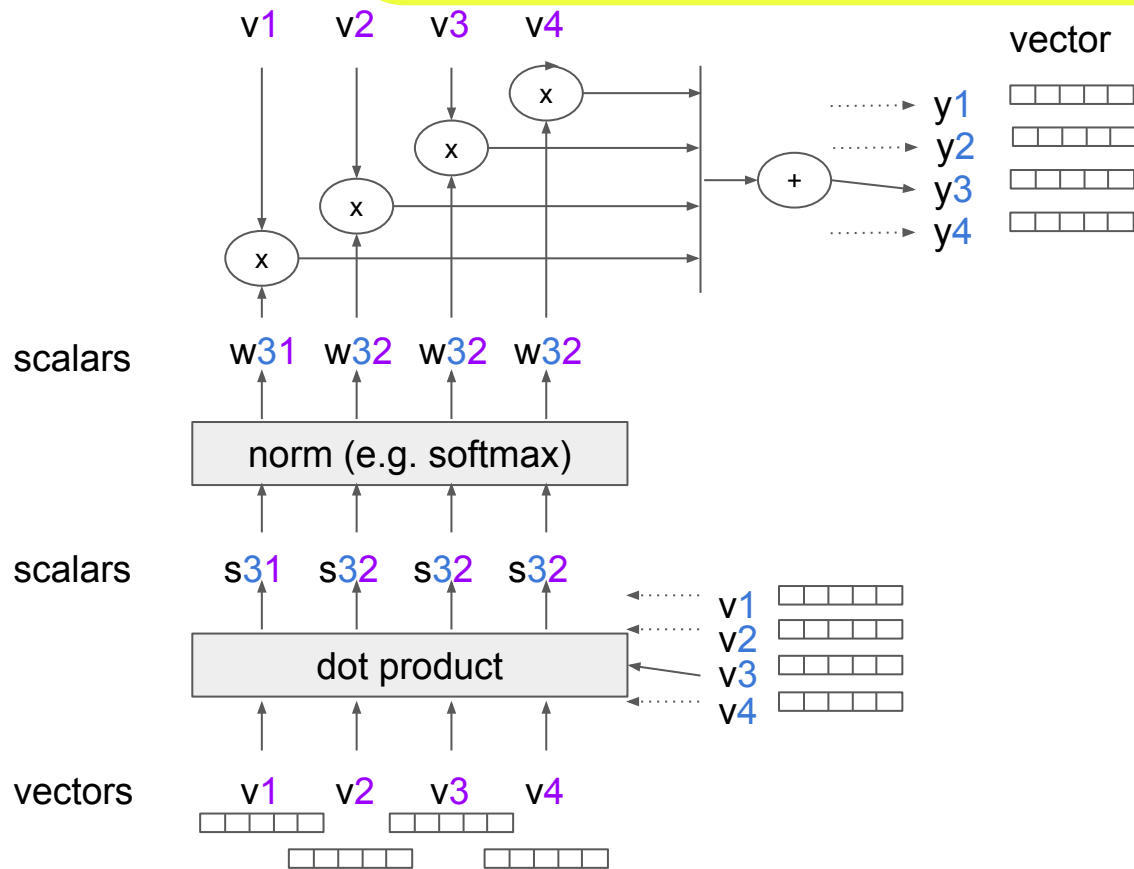
$w11$
 $w12$
 $w13$
 $w14$

$w1$

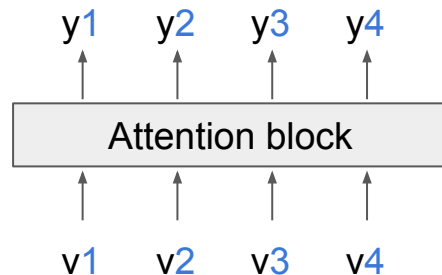
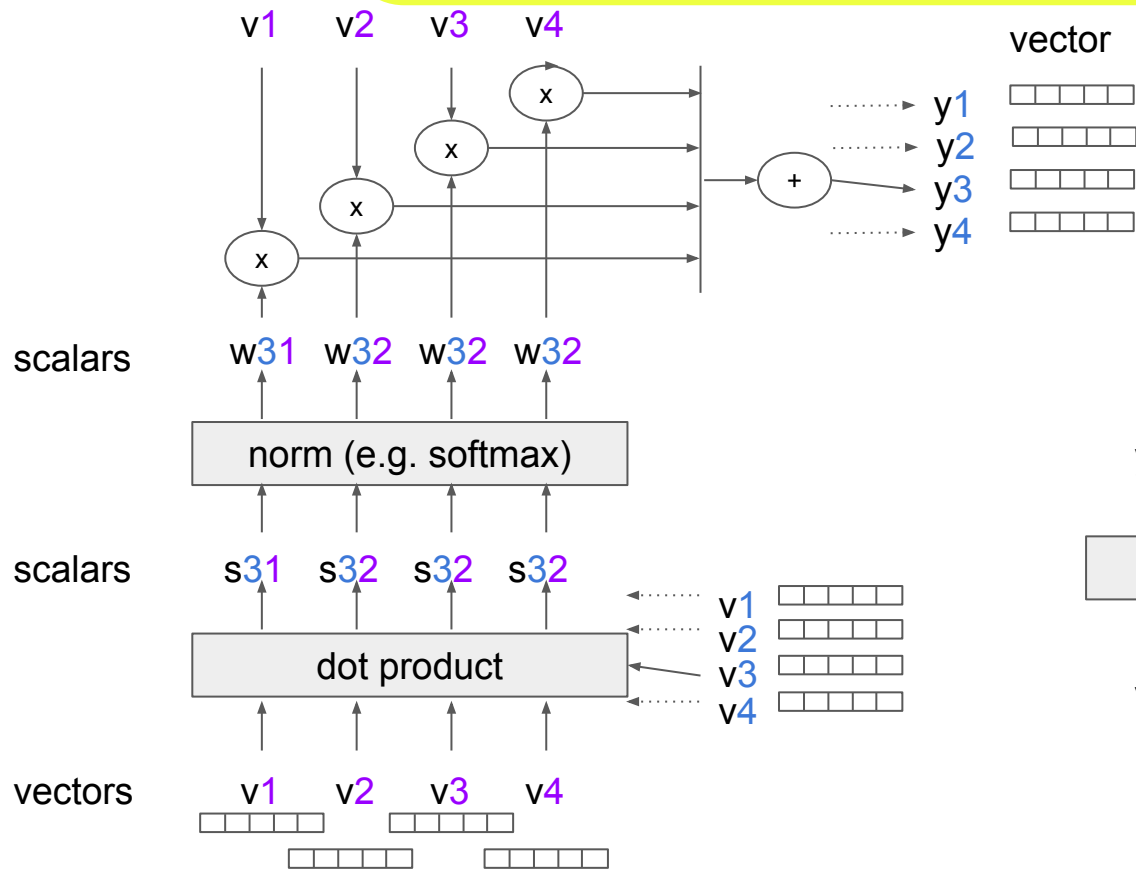
Self-Attention



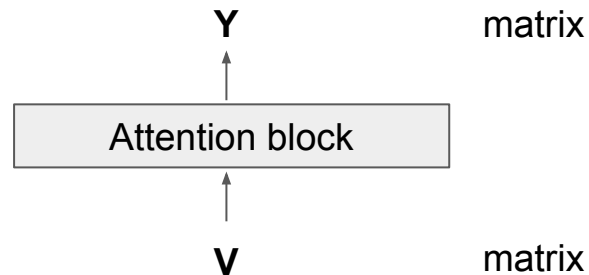
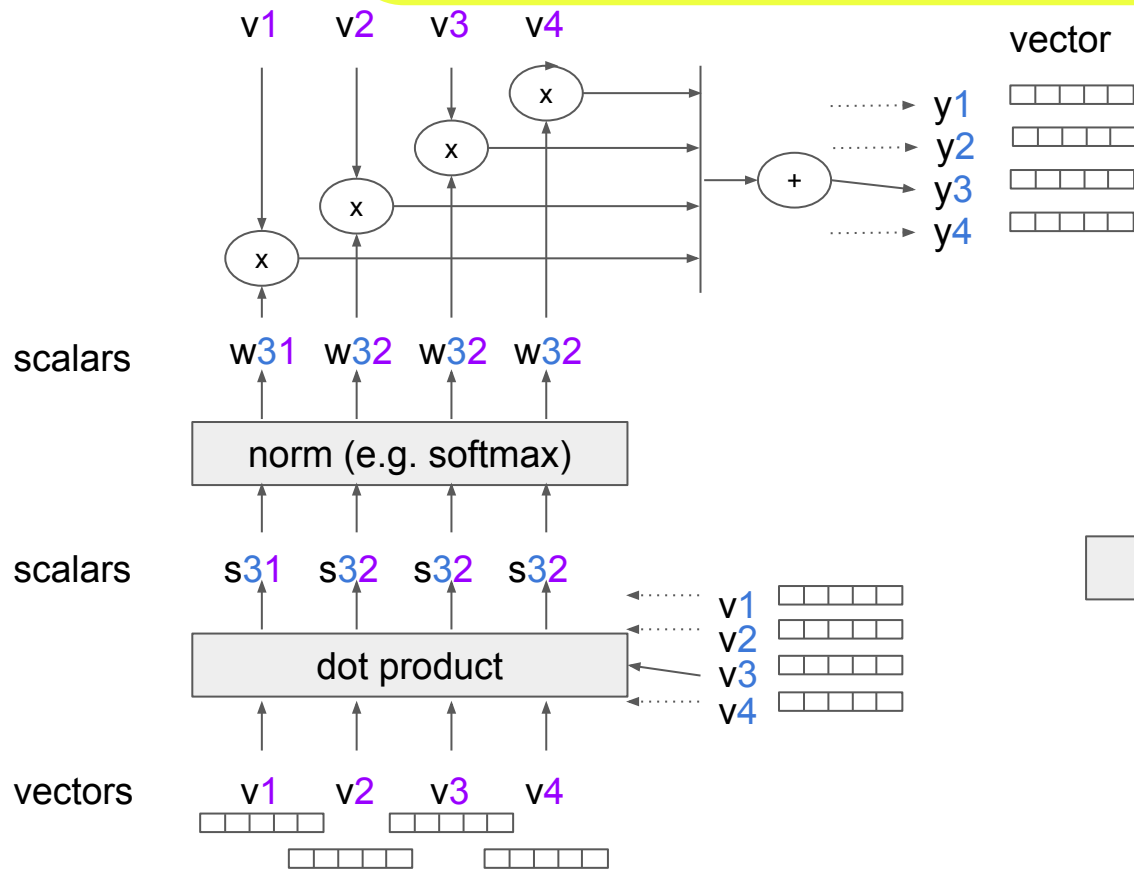
Self-Attention



Self-Attention



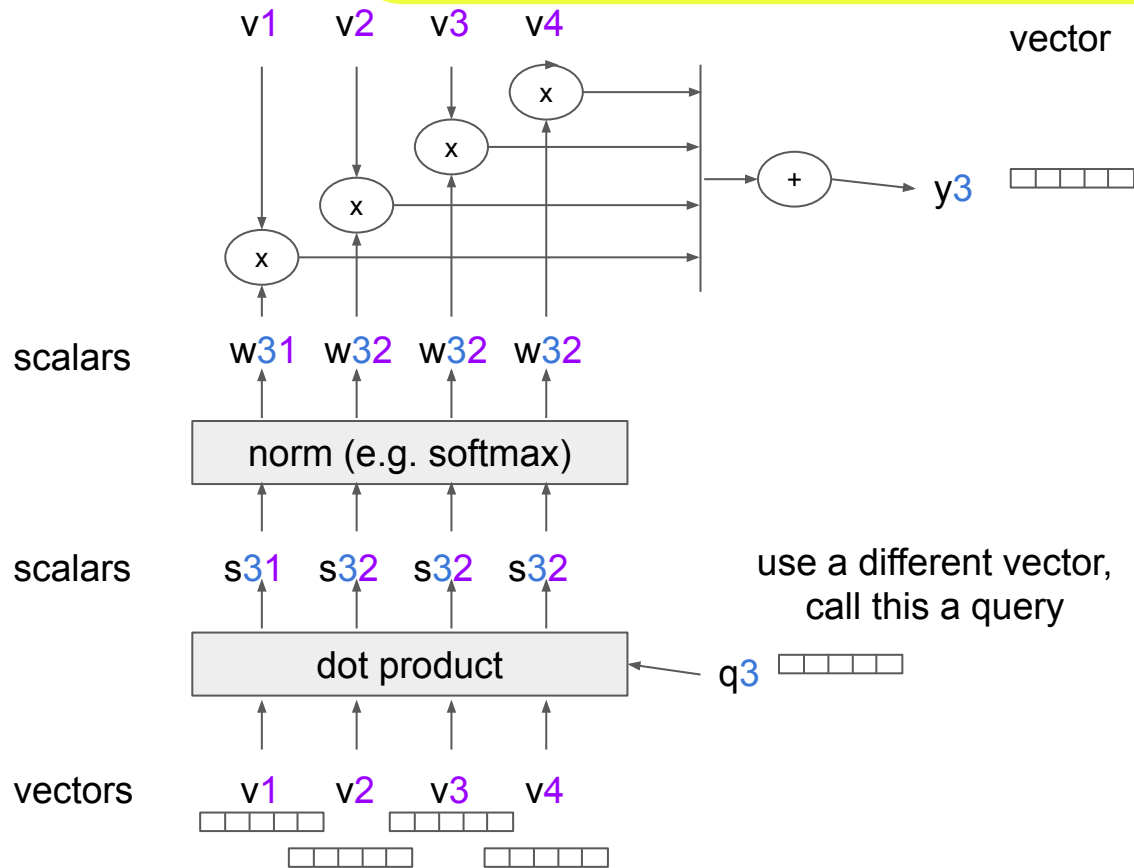
Self-Attention



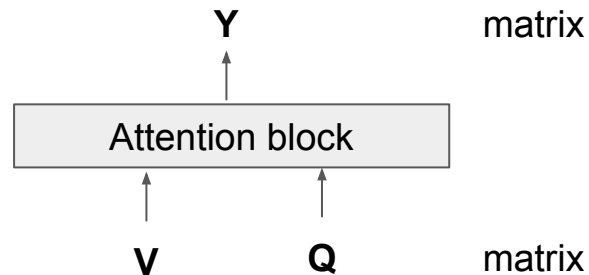
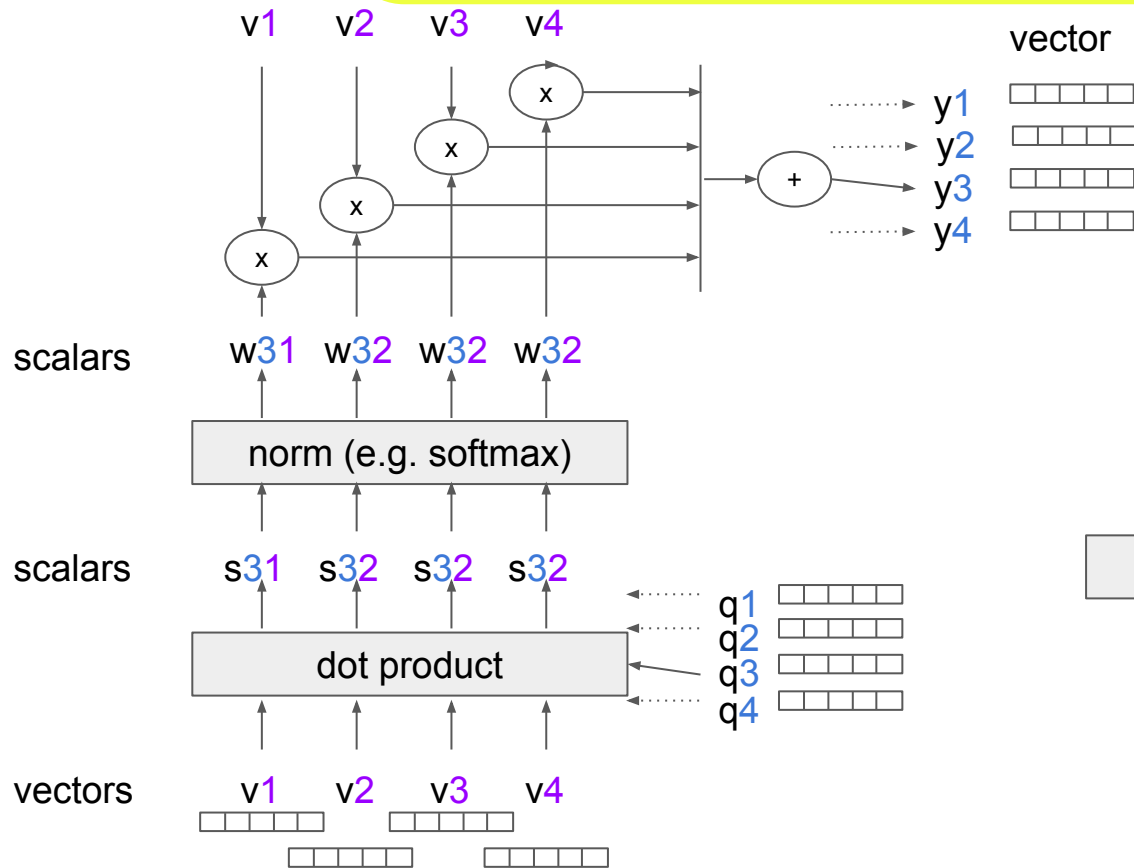
Seq2seq with Attention

- Attention provides a solution to the bottleneck problem.
- Core idea: on each step of the decoder, use direct connection to the encoder to focus on a particular part of the source sequence
- First we will show via diagram (no equations), then we will show with equations
- **Self-Attention (Where we were):** Looks inward: connects words within the same sentence to build context.
- **Cross / Seq2Seq Attention (Where we are going):** Looks across: connects the output directly back to the input, step by step.

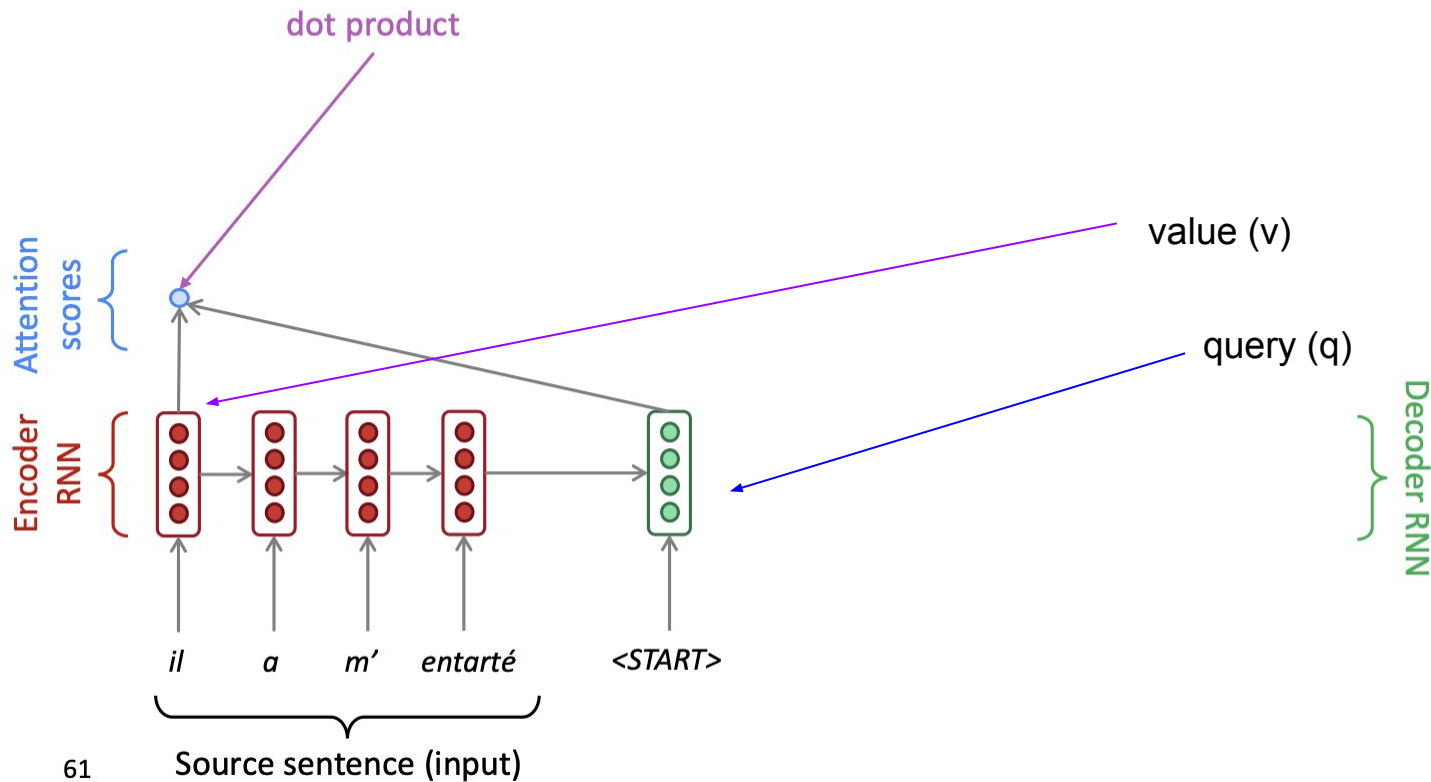
Attention



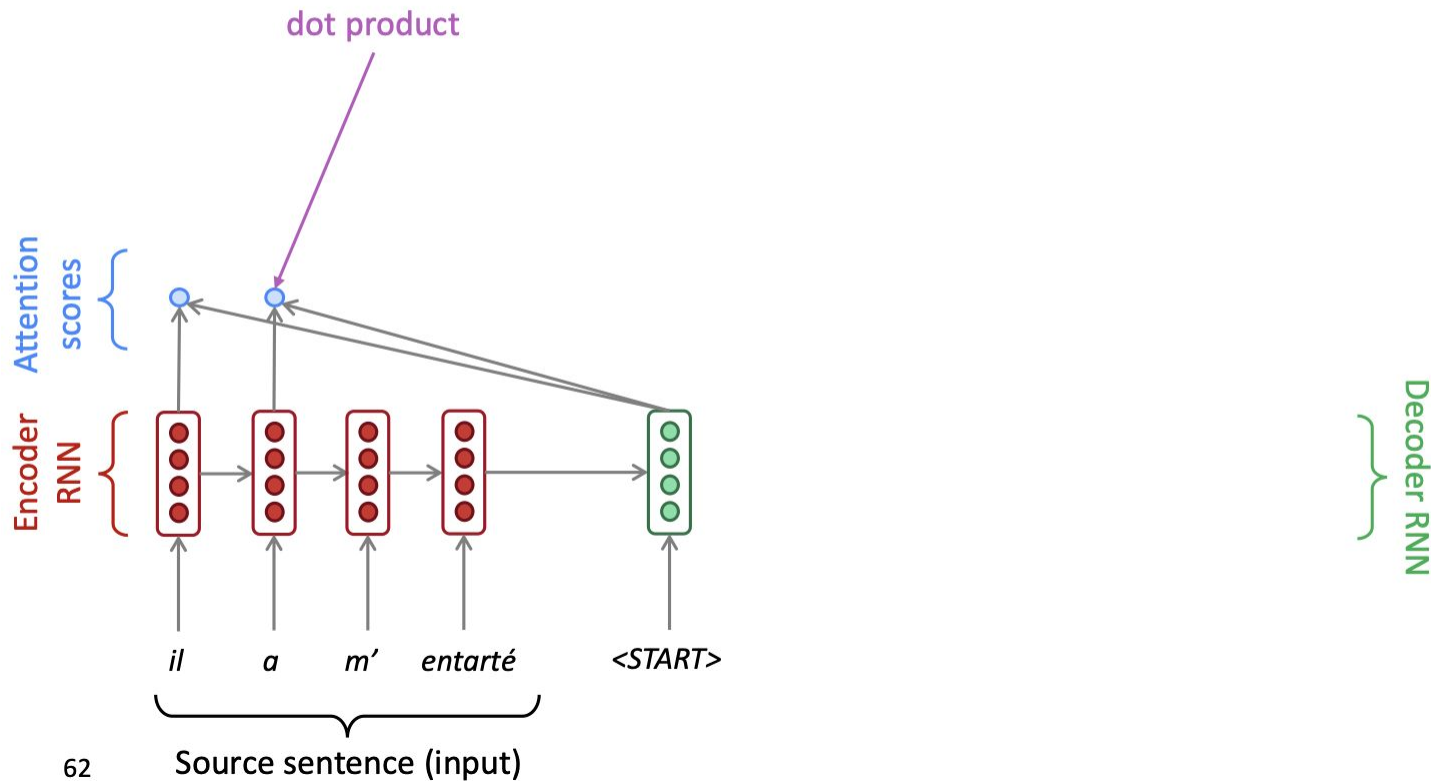
Attention



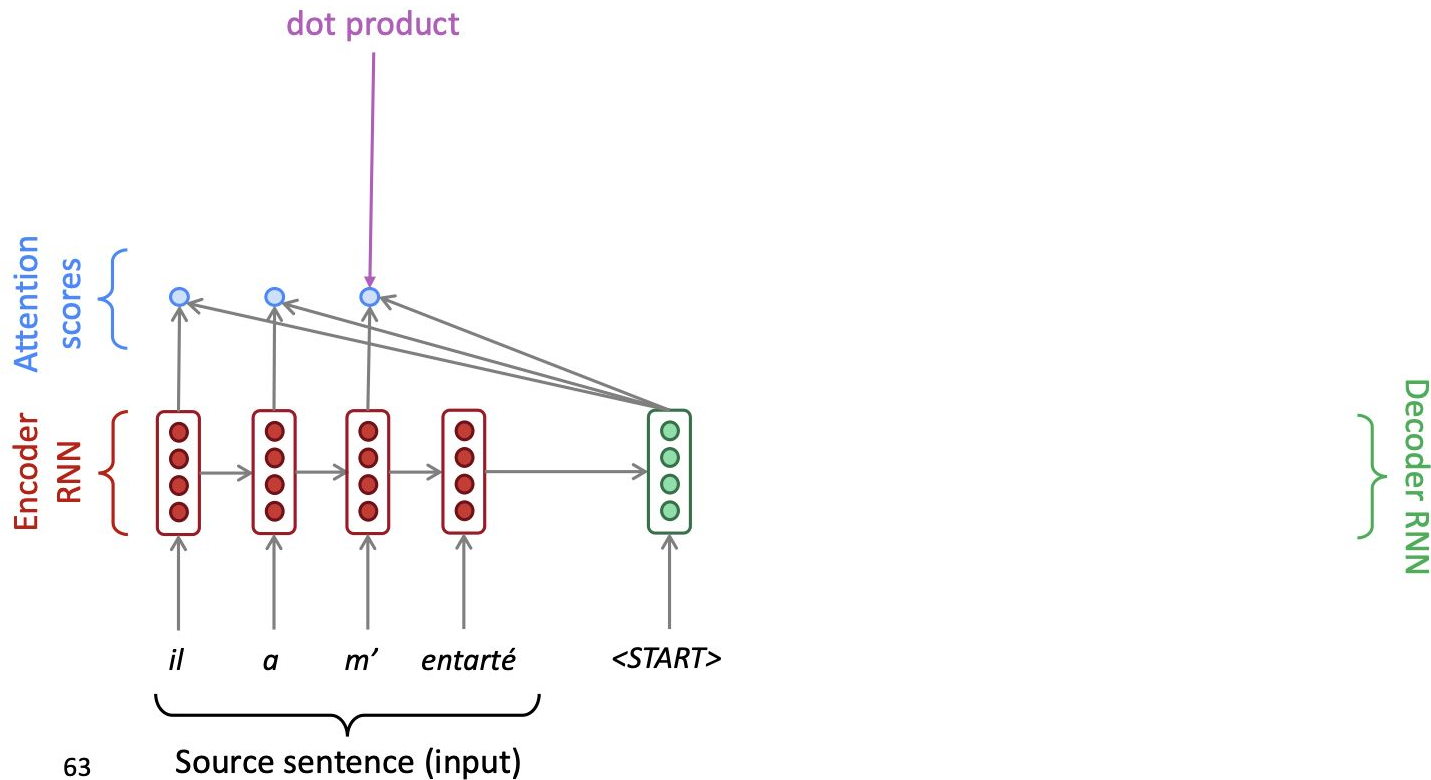
Seq2seq with Attention



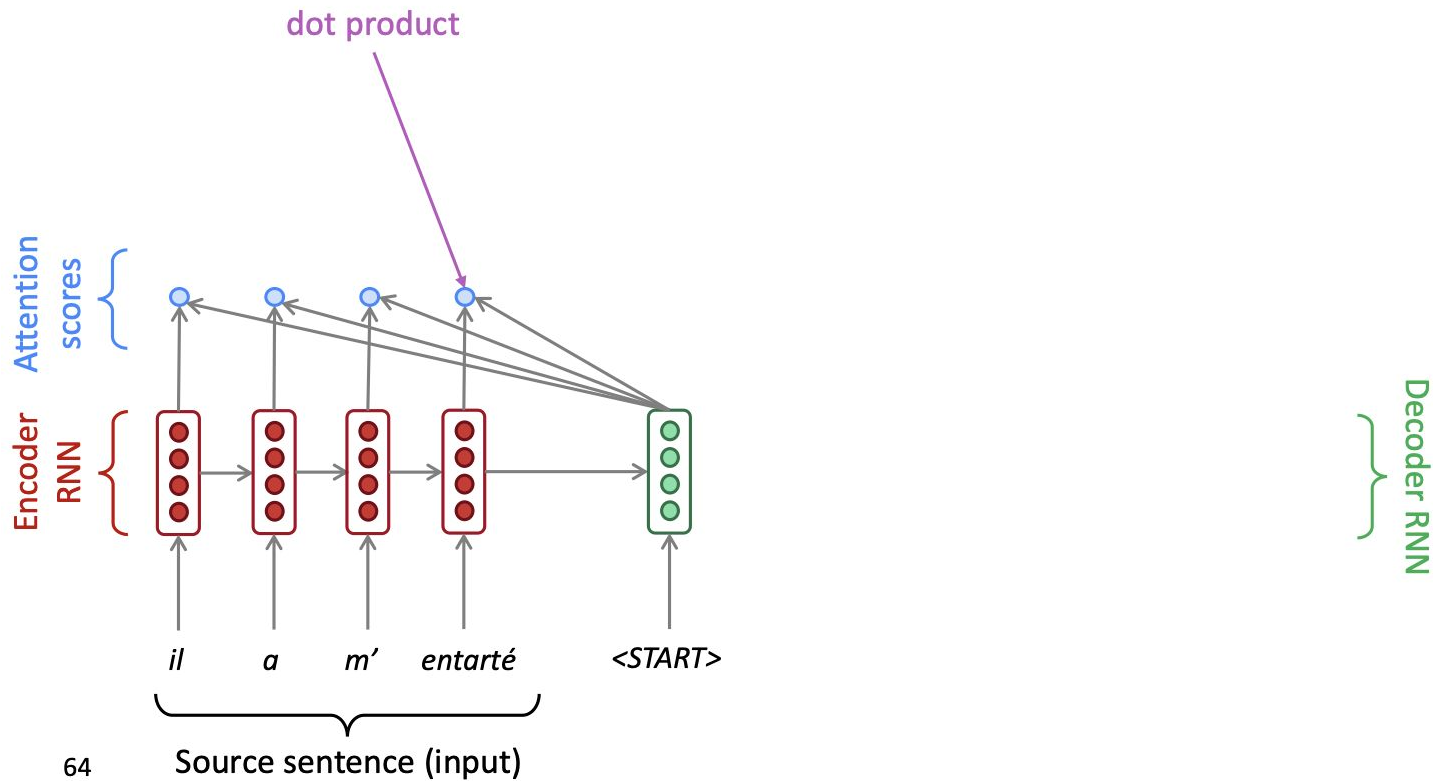
Seq2seq with Attention



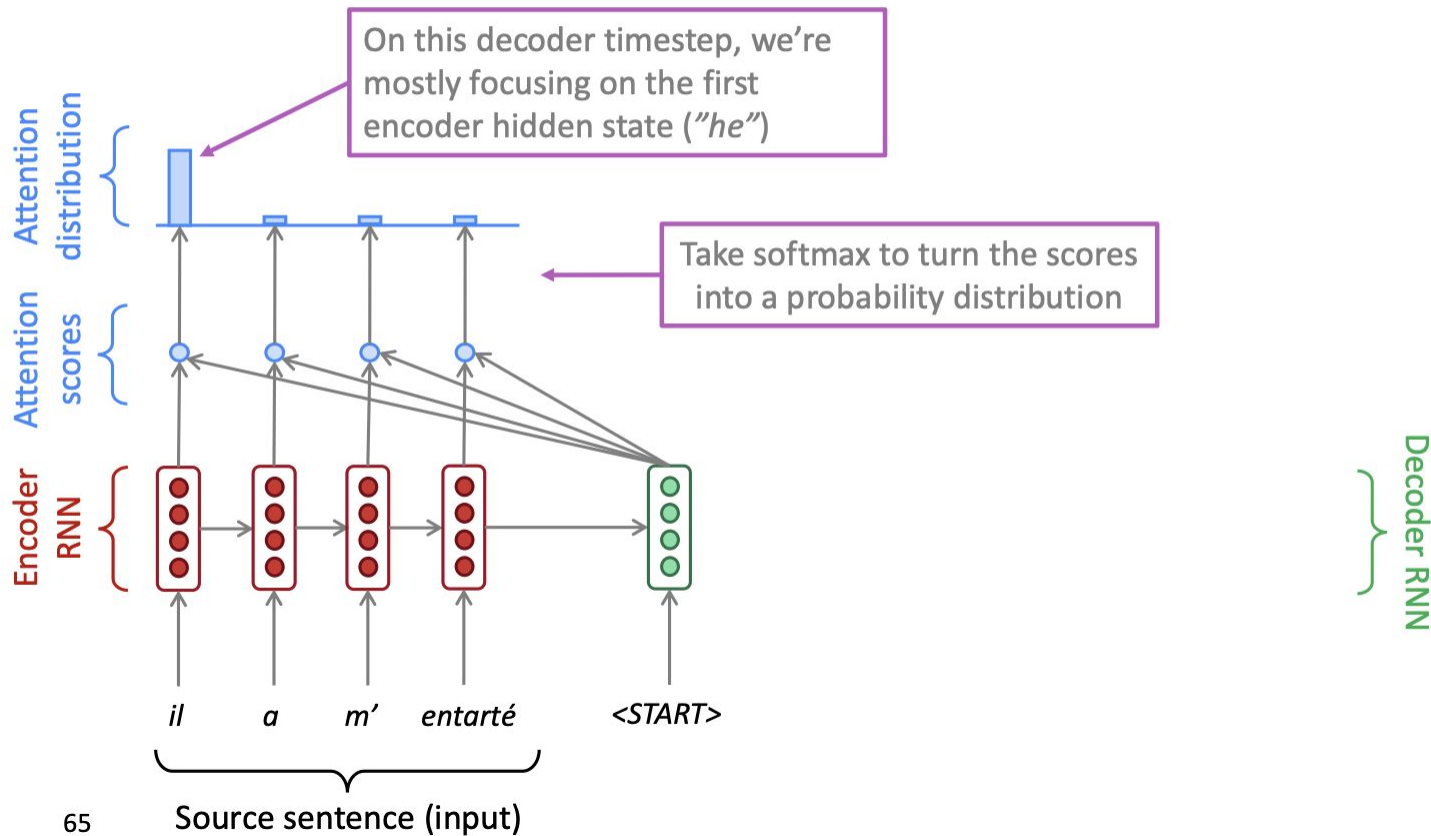
Seq2seq with Attention



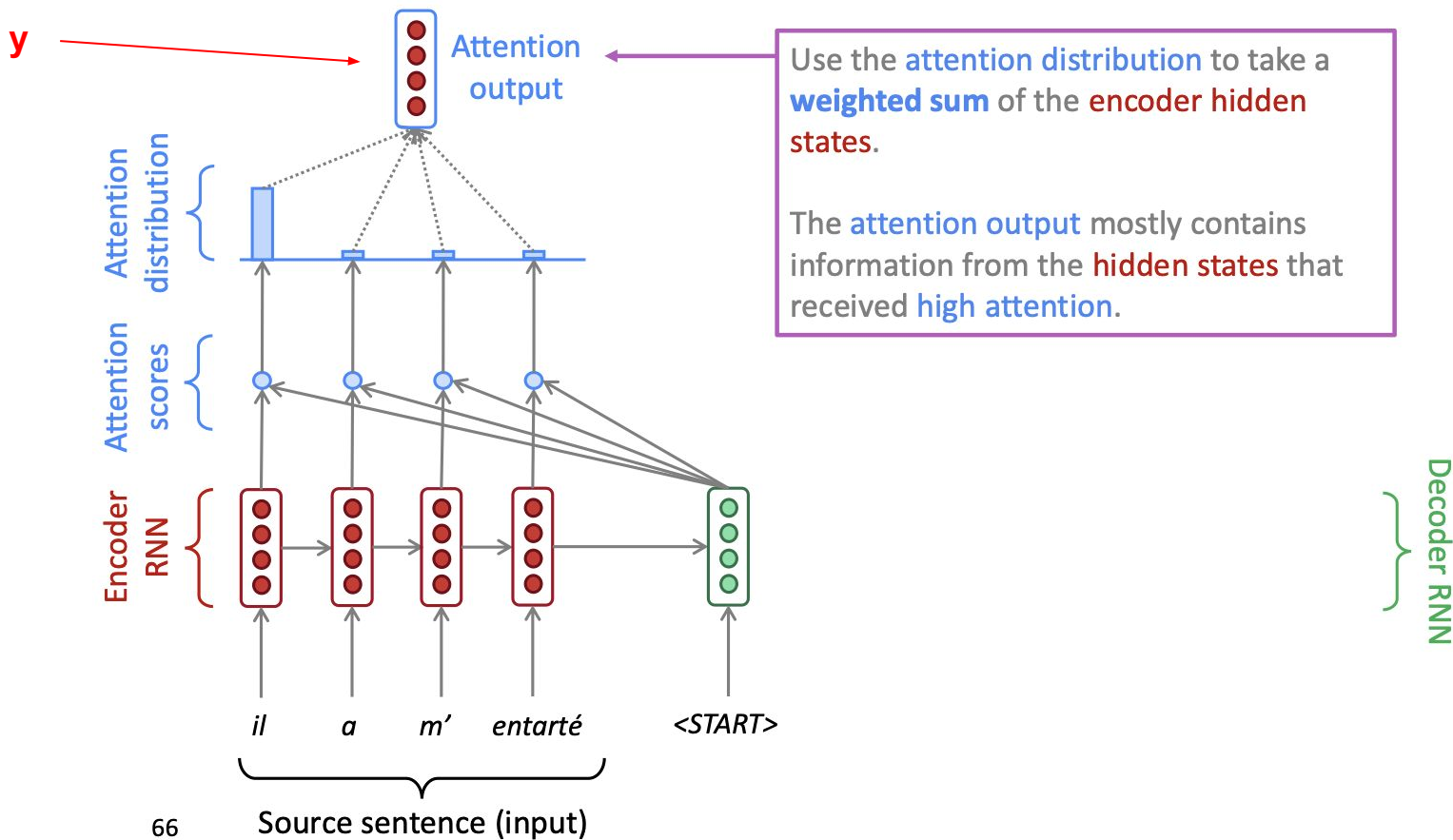
Seq2seq with Attention



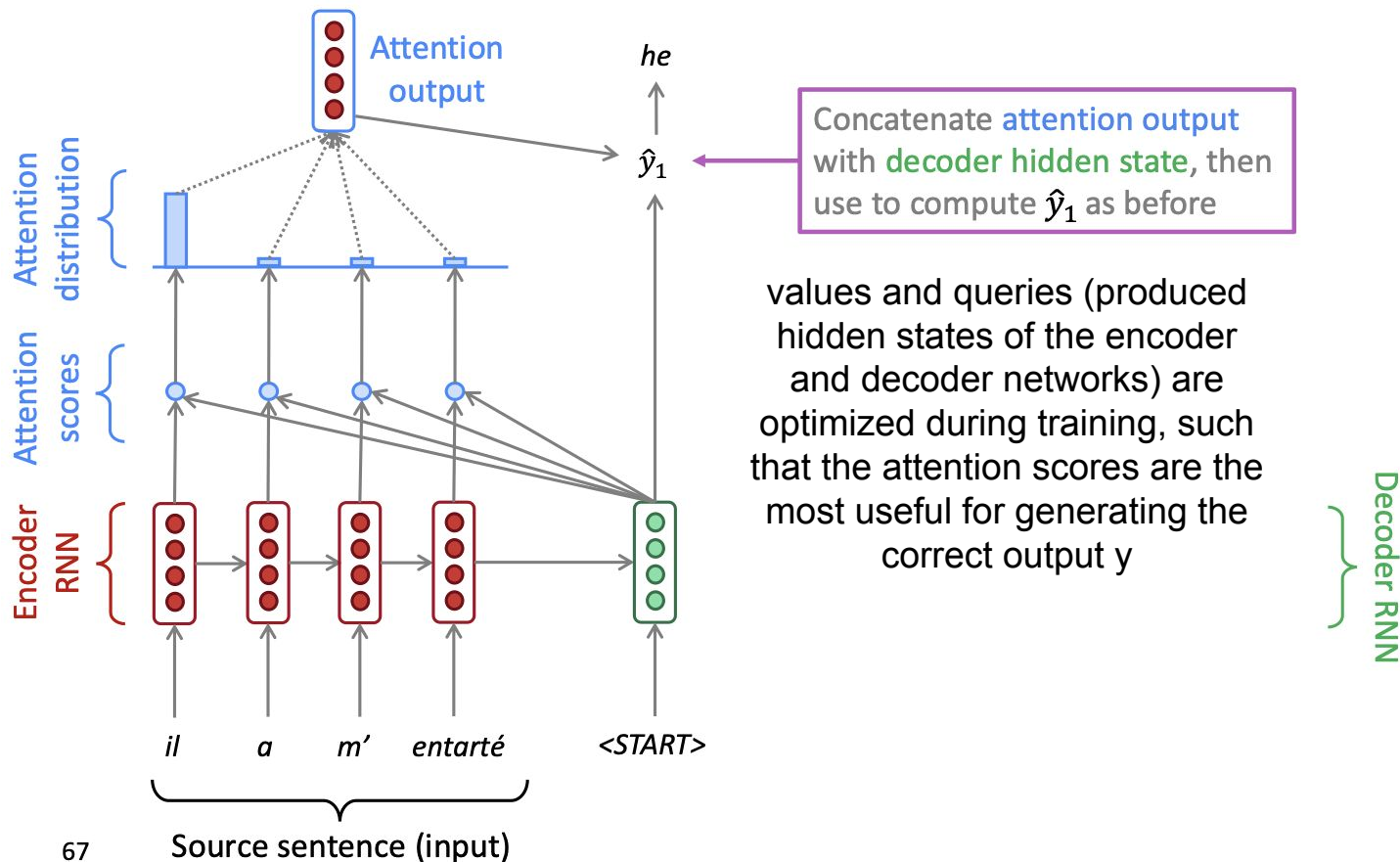
Seq2seq with Attention



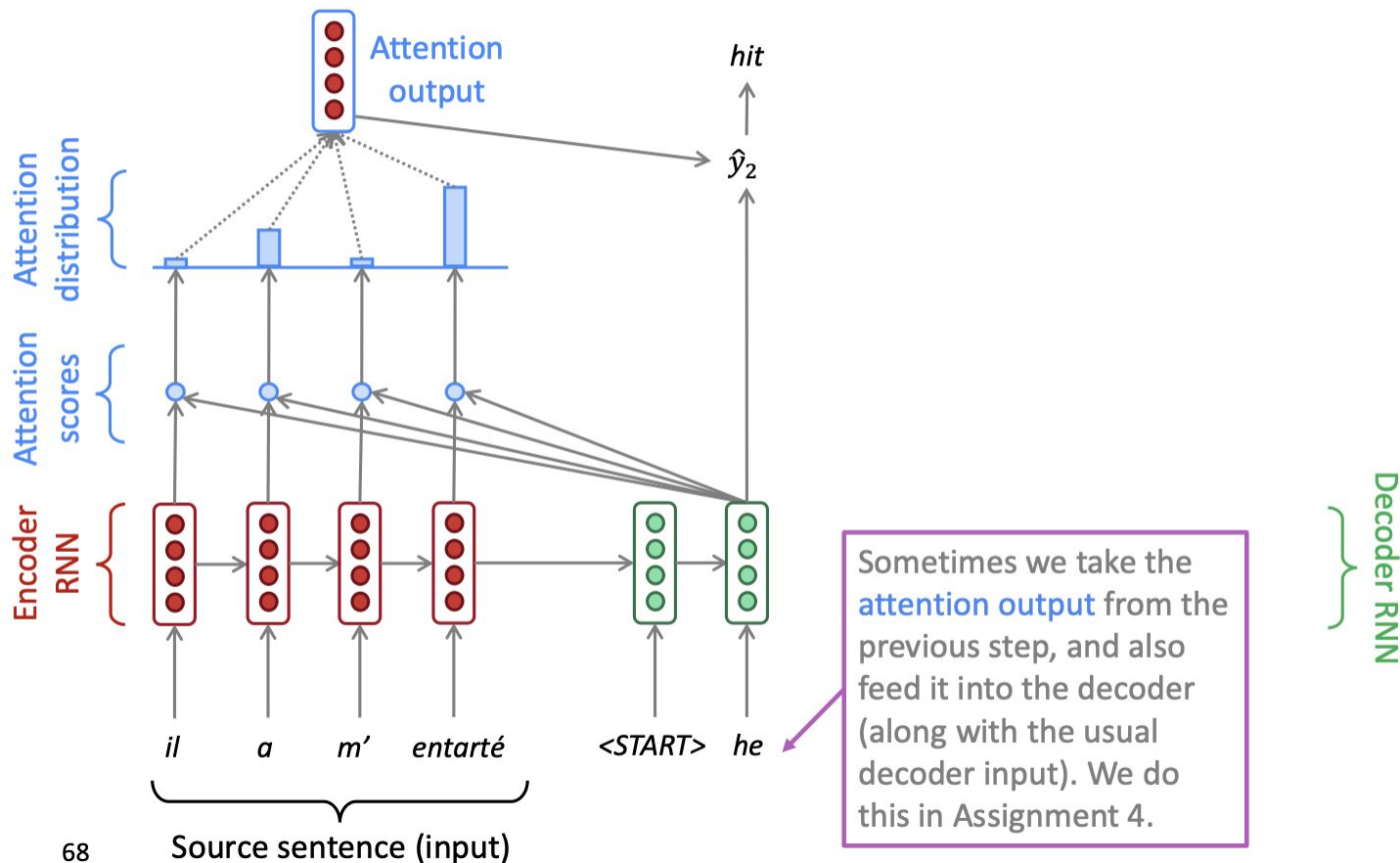
Seq2seq with Attention



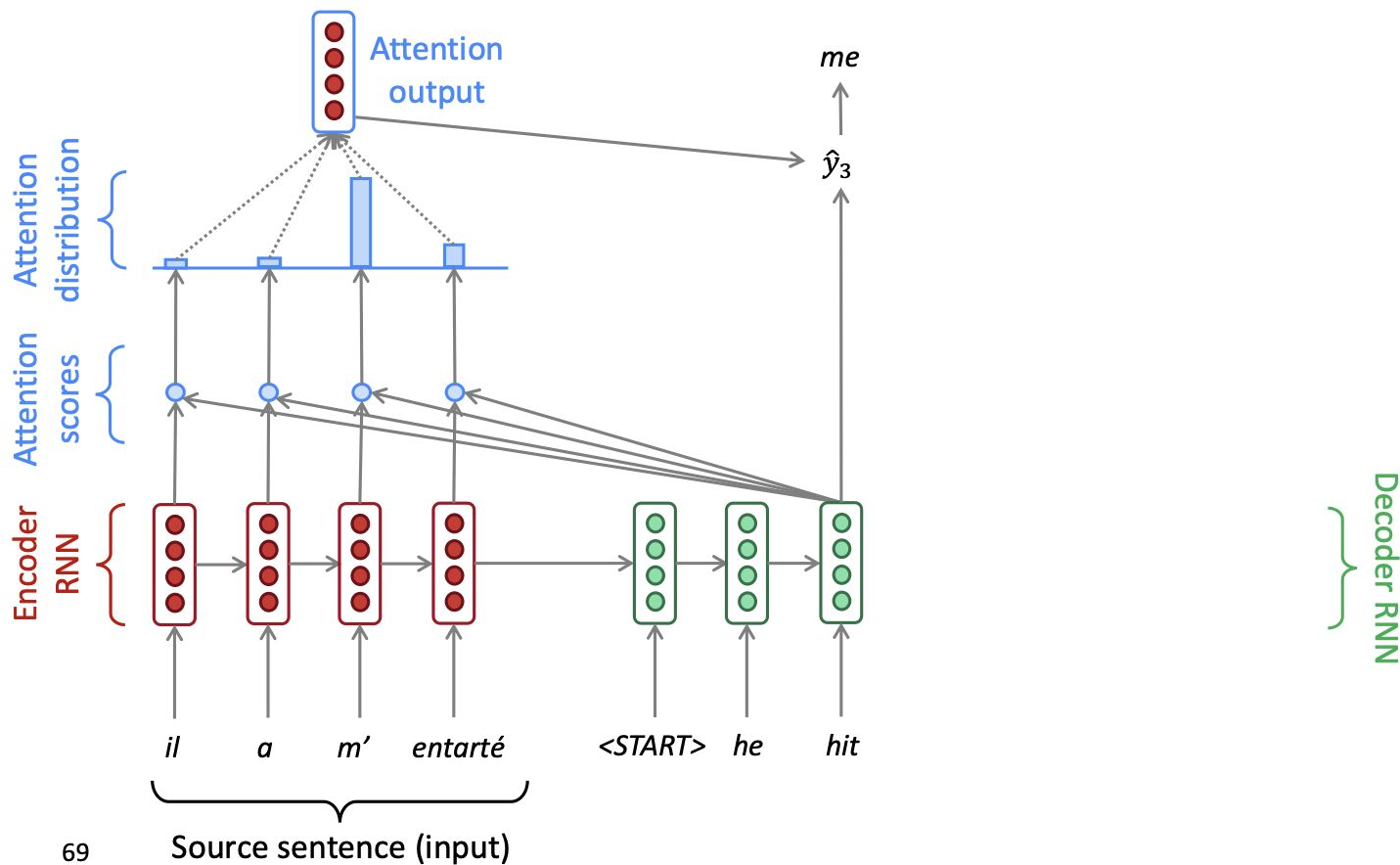
Seq2seq with Attention



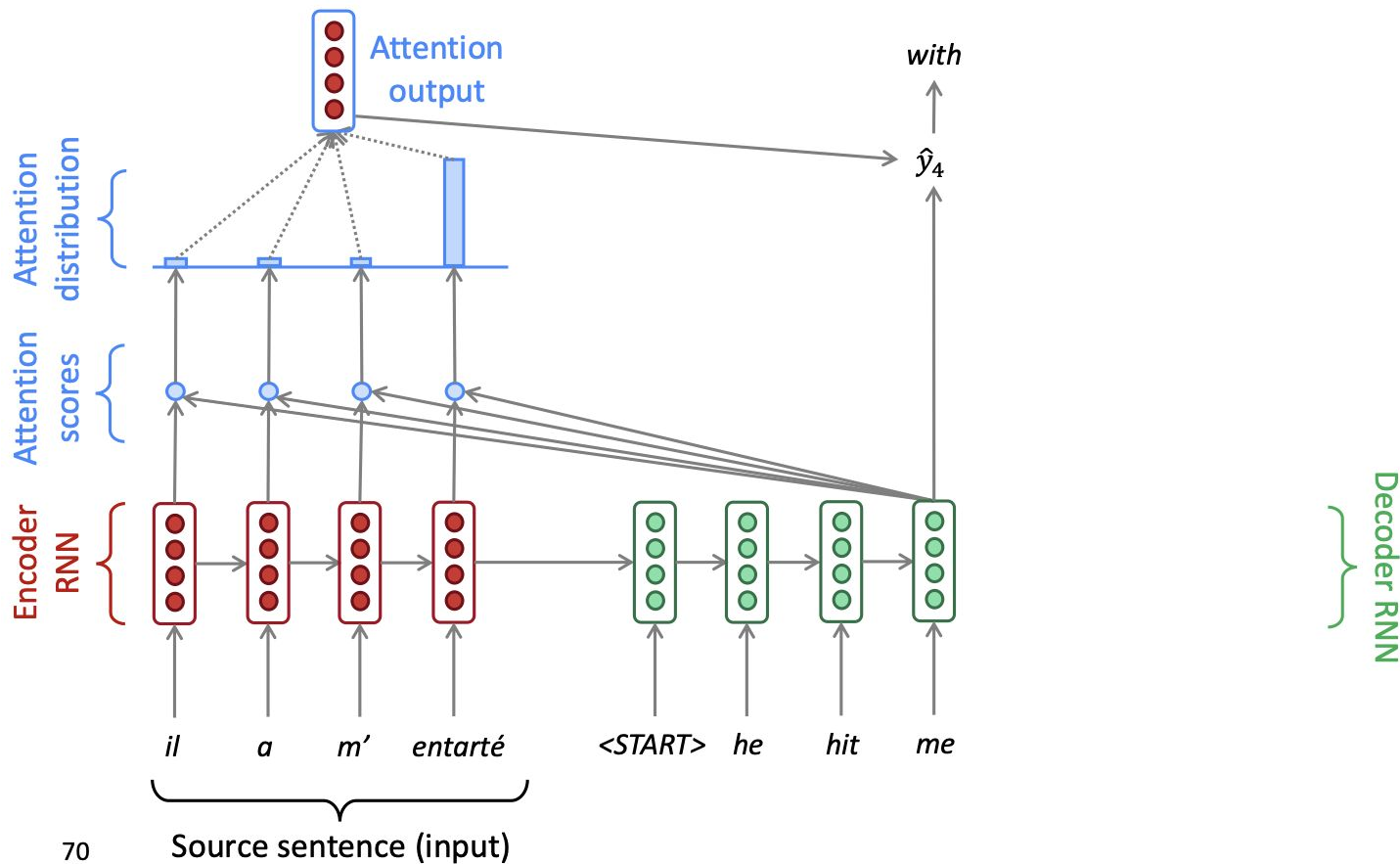
Seq2seq with Attention



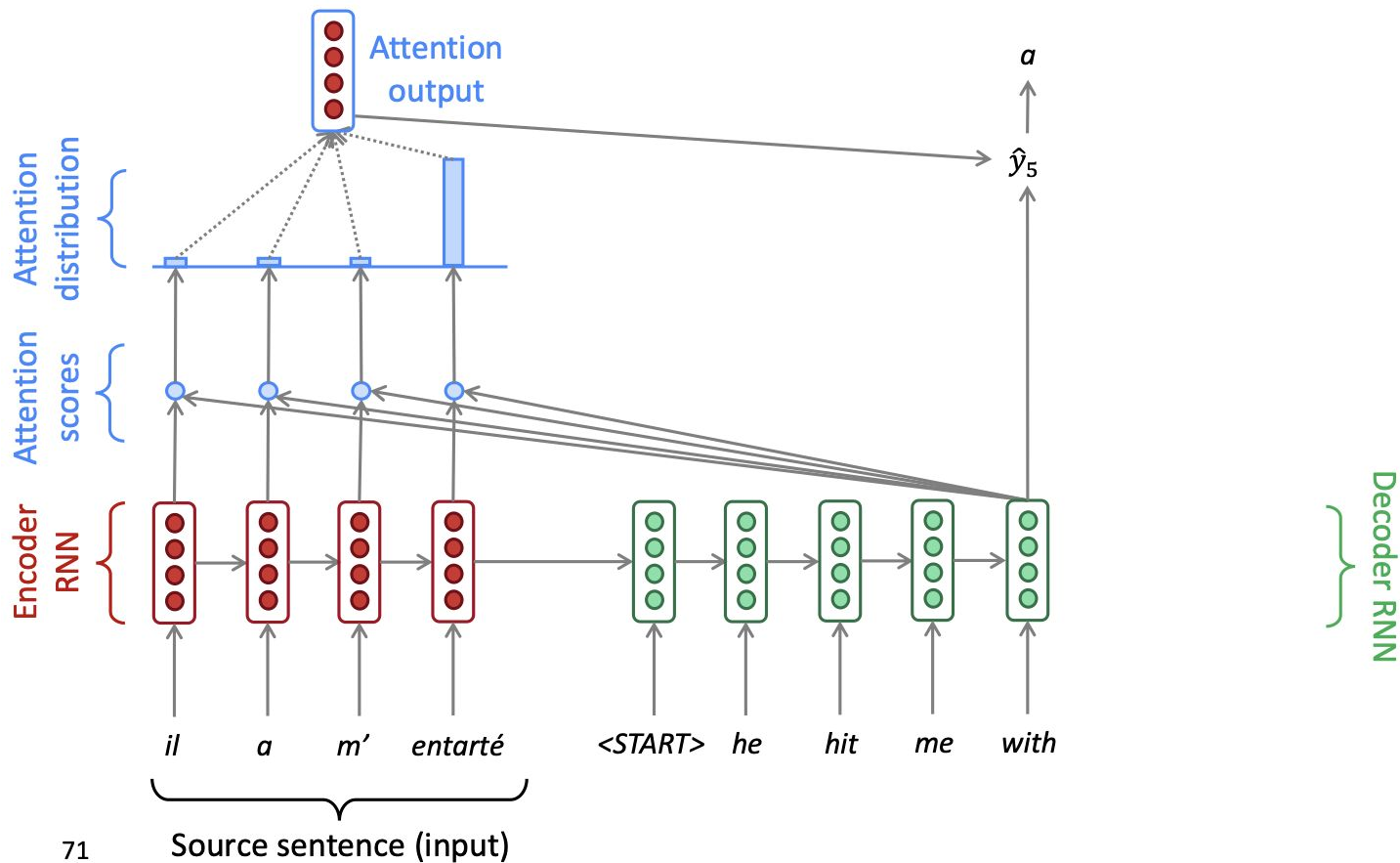
Seq2seq with Attention



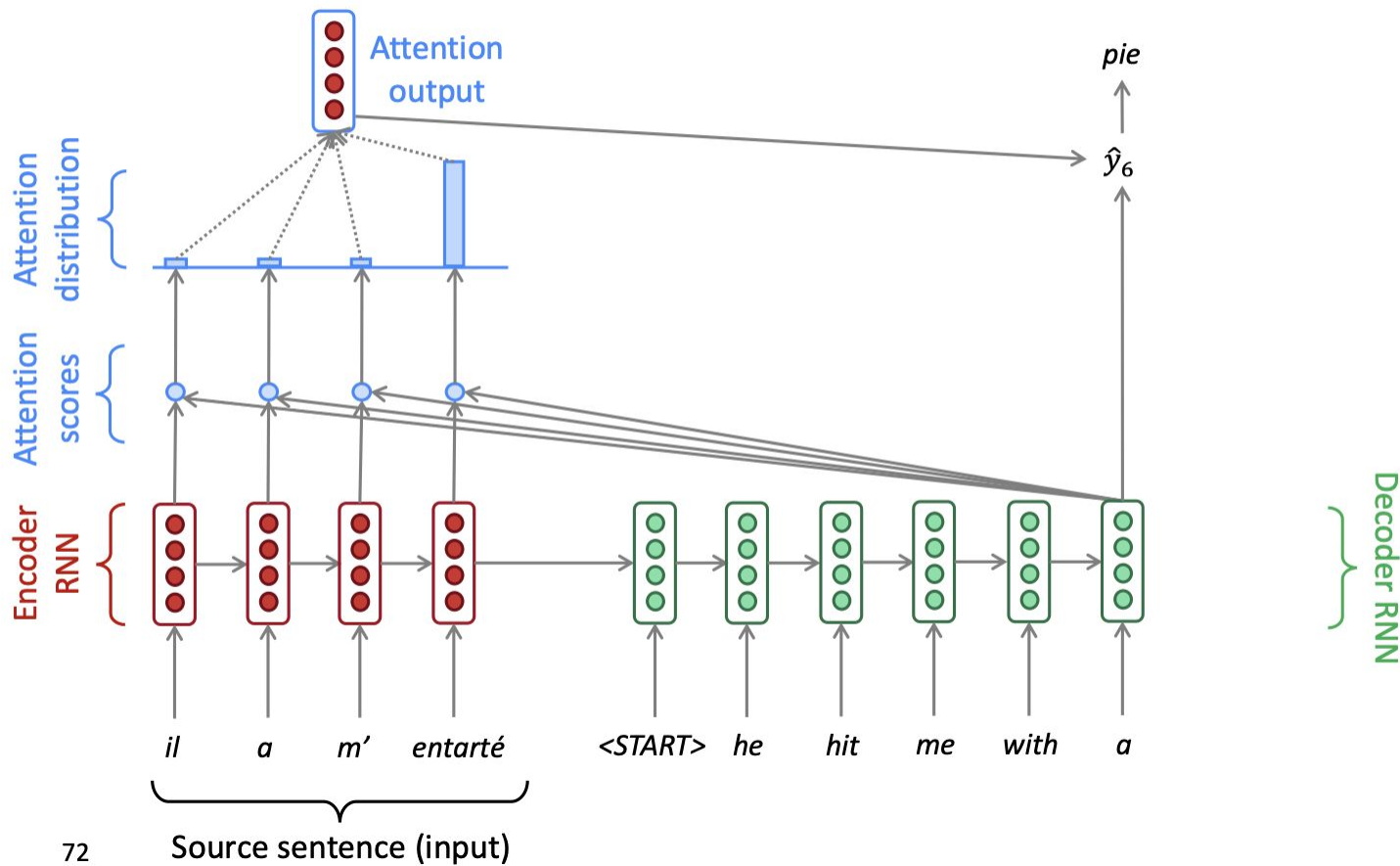
Seq2seq with Attention



Seq2seq with Attention



Seq2seq with Attention



Seq2seq with Attention

- We have encoder hidden states $h_1, \dots, h_n \in \mathbb{R}^h$ ← values
- On timestep t , we have decoder hidden state $s_t \in \mathbb{R}^h$ ← query
- We get the attention scores e^t for this step:

$$e^t = [s_t^T h_1, \dots, s_t^T h_N] \in \mathbb{R}^N$$

- We take softmax to get the attention distribution α^t for this step (this is a probability distribution and sums to 1)

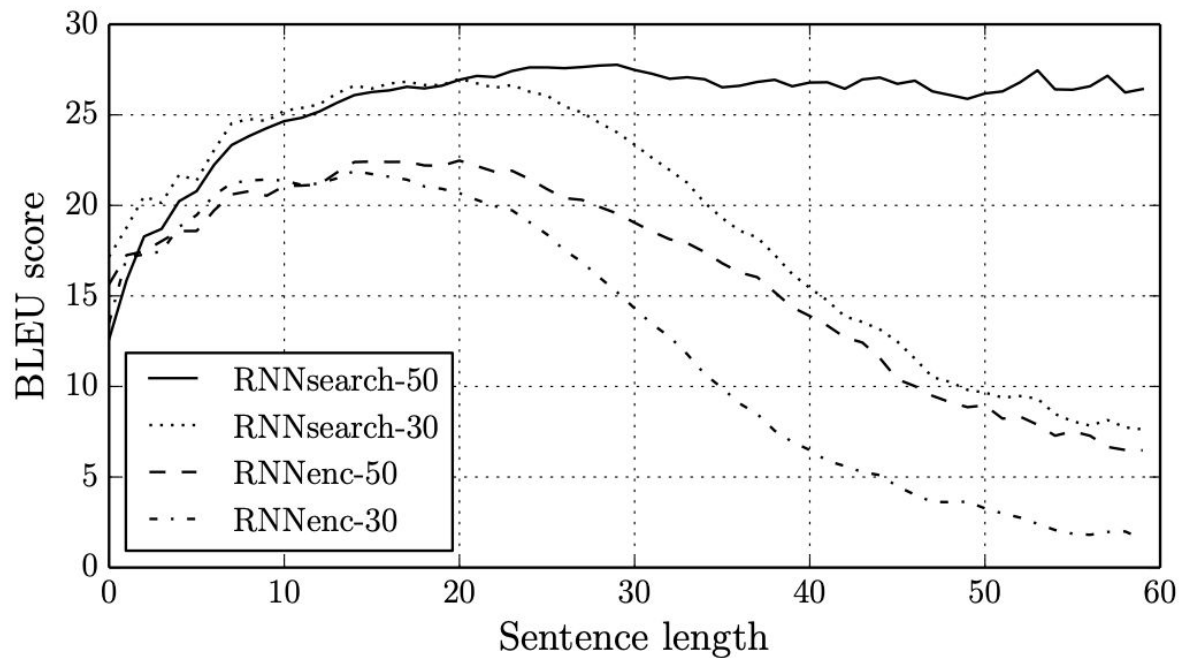
$$\alpha^t = \text{softmax}(e^t) \in \mathbb{R}^N$$

- We use α^t to take a weighted sum of the encoder hidden states to get the attention output $\mathbf{a}_t$

$$\mathbf{a}_t = \sum_{i=1}^N \alpha_i^t h_i \in \mathbb{R}^h$$

- Finally we concatenate the attention output $\mathbf{a}_t$ with the decoder hidden state $\mathbf{s}_t$ and proceed as in the non-attention seq2seq model $[\mathbf{a}_t; \mathbf{s}_t] \in \mathbb{R}^{2h}$

Seq2seq with Attention



Improvements of BLEU scores with attention

Attention

- Attention significantly improves NMT performance
 - It's very useful to allow decoder to focus on certain parts of the source
- Attention solves the bottleneck problem
 - Attention allows decoder to look directly at source; bypass bottleneck
- Attention helps with vanishing gradient problem
 - Provides shortcut to faraway states
- Attention provides some interpretability
 - By inspecting attention distribution, we can see what the decoder was focusing on
 - We get (soft) alignment for free!
 - This is cool because we never explicitly trained an alignment system
 - The network just learned alignment by itself

	he	hit	me	with	a	pie
il						
a						
m'						
entarté						

Attention

- **Attention is a general Deep Learning technique**
- We've seen that attention is a great way to improve the sequence-to-sequence model for Machine Translation.
- However: You can use attention in many architectures (not just seq2seq) and many tasks (not just machine translation)
- More general definition of attention:
 - Given a set of vector values, and a vector query, attention is a technique to compute a weighted sum of the values, dependent on the query.
- We sometimes say that the query attends to the values.
- For example, in the seq2seq + attention model, each decoder hidden state (query) attends to all the encoder hidden states (values).

Attention

More general definition of attention:

- Given a set of vector values, and a vector query, attention is a technique to compute a weighted sum of the values, dependent on the query.

Intuition:

- The weighted sum is a selective summary of the information contained in the values, where the query determines which values to focus on.
- Attention is a way to obtain a fixed-size representation of an arbitrary set of representations (the values), dependent on some other representation (the query).

Attention variants

- There are several attention variants
- We have some values $h_1, \dots, h_n \in \mathbb{R}^{d_1}$ and a query $s \in \mathbb{R}^{d_2}$
- Attention always involves:
 1. Computing the attention scores $e \in \mathbb{R}^N$ (There are multiple ways to do this)
 2. Taking softmax to get attention distribution α :
$$\alpha = \text{softmax}(e) \in \mathbb{R}^N$$
 3. Using attention distribution to take weighted sum of values:
$$a = \sum_{i=1}^N \alpha_i h_i \in \mathbb{R}^{d_1}$$

thus obtaining the attention output a (sometimes called the context vector)

Attention variants

There are several ways you can compute e from $h_1, \dots, h_N$ and s :

- Basic dot-product attention: $e_i = s^T h_i \in \mathbb{R}$
 - Note: this assumes $d_1 = d_2$
 - This is the version we saw earlier
- Multiplicative attention: $e_i = s^T W h_i \in \mathbb{R}$
 - Where $W \in \mathbb{R}^{d_2 \times d_1}$ is a weight matrix
- Additive attention: $e_i = v^T \tanh(W_1 h_i + W_2 s) \in \mathbb{R}$
 - Where $W_1 \in \mathbb{R}^{d_3 \times d_1}$, $W_2 \in \mathbb{R}^{d_3 \times d_2}$ are weight matrices and $v \in \mathbb{R}^{d_3}$ is a weight vector.
 - d_3 (the attention dimensionality) is a hyperparameter

Attention variants

There are several ways you can compute e from $h_1, \dots, h_N$ and s :

- Basic dot-product attention: $e_i = s^T h_i \in \mathbb{R}$
 - Note: this assumes $d_1 = d_2$
 - This is the version we saw earlier
- Multiplicative attention: $e_i = s^T W h_i \in \mathbb{R}$
 - Where $W \in \mathbb{R}^{d_2 \times d_1}$ is a weight matrix
- Additive attention: $e_i = v^T \tanh(W_1 h_i + W_2 s) \in \mathbb{R}$
 - Where $W_1 \in \mathbb{R}^{d_3 \times d_1}$, $W_2 \in \mathbb{R}^{d_3 \times d_2}$ are weight matrices and $v \in \mathbb{R}^{d_3}$ is a weight vector.
 - d_3 (the attention dimensionality) is a hyperparameter

These weights can be learnt during backpropagation

It turns out...

Attention Is All You Need

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin, <https://arxiv.org/abs/1706.03762>